



Gestione degli errori

Elaborazioni non valide

Lezioni 14 15-
Corso 1



Errori ed eccezioni

lezione del 21 aprile 12:02



- Tutte le computazioni possono fallire prematuramente
 - Impedendo che possa essere restituito il valore atteso o che vengano eseguiti tutti gli effetti collaterali richiesti
- Questi fallimenti possono essere dovuti a cause differenti:
 - errori di programmazione
 - errori causati dall'input fornito dall'utente finale
 - errori causati dal sistema per mancanza di risorse
- Alcuni sono facilmente prevedibili (argomenti illeciti, conversione da testo a numero, ...)
- Altri sono dovuti a limiti del sistema di elaborazione (memoria/spazio su disco esauriti, malfunzionamento della rete o di altre periferiche, ...) che raramente sono presi in considerazione perché il programmatore tende a semplificare la realtà
- Indipendentemente dalle cause che li hanno prodotti, i fallimenti possono essere catalogati in due gruppi principali
 - Malfunzionamenti **recuperabili** - quelli che non hanno compromesso lo stato del programma, per cui è possibile attivare una strategia di ripristino
 - Malfunzionamenti **non recuperabili** - quelli che causano un'alterazione imprevedibile dello stato o che indicano l'impossibilità di procedere ulteriormente nella computazione

Errori ed eccezioni

- Nei casi irrecuperabili, occorre terminare il processo
 - Eventualmente eseguendo qualche operazione di pulizia sull'ambiente esterno
- Negli altri, occorre mettere in atto una strategia di ripristino dello stato
 - Ritentare l'operazione, richiedere l'intervento dell'utente/amministratore, utilizzare una strategia alternativa, ...
- Non è detto che il punto in cui si verifica il fallimento possieda abbastanza contesto per discriminare come comportarsi
 - Occorre fare in modo che, in caso di fallimento della computazione all'interno di una funzione, il controllo torni al suo chiamante, corredato di una opportuna descrizione di quanto successo
- Questo requisito tende ad aumentare la complessità del codice, introducendo rami diversi (if/else, switch/case, match...) lungo i quali la computazione può procedere, favorendo l'introduzione di errori logici legati alla quantità di dettagli cui occorre badare
 - Per questo motivo, i linguaggi "moderni" includono particolari costrutti a supporto della gestione delle eccezioni

Supporto sintattico alla gestione degli errori

- Il linguaggio C non fornisce alcun supporto sintattico alla modellazione degli errori
 - Lasciando completamente al programmatore la responsabilità di gestire la situazione
- **Il linguaggio C++** (come molti altri linguaggi, come Java, C#, JavaScript, ...) **introduce il concetto di eccezione**
 - Offre le parole-chiave `try/catch/throw/finally` per esprimere la logica di notifica e gestione
 - Si prova (try) a fare l'operazione,
 - Impone una particolare struttura del contesto di esecuzione per supportare tale genere di astrazione (ad es. come è fatto lo stack e quali sono i registri del processore che sto usando) => crea problemi per i moduli Kernel che non hanno un meccanismo per la gestione delle eccezioni
 - Kernel di Linux non è fatto per gestire codice C++ che impone dei vincoli sul contesto di esecuzione
- **Rust non utilizza il concetto di eccezione**, ma offre i tipi algebrici `Result<T,E>` (funzione termina con successo o con errore) e `Option<T>` (che fornisce il risultato oppure informa che non c'è risultato) per esprimere gli esiti delle computazioni
 - Offre inoltre la macro `panic!(...)` per forzare l'interruzione del thread corrente producendo una descrizione testuale di quanto successo, se corrisponde al thread principale, l'intero processo viene chiuso, stampando una stringa su `stderr`.

r1 è il valore di ritorno della funzione open ed è di tipo Result, ovvero un enum. Questo deve essere spaccettato tramite la clausola match -> pattern matching. QUesto è l'uso più classico, per analizzare quale variante corrisponde ad un risultato

ciò che è stato restituito dalla funzione read to string viene passando al chiamante nella forma OK(s)

Gestioni delle eccezioni in Rust

file è un puntatore ad un file, per cui quando esco dalla procedura, la variabile viene deallocata e tutti i suoi dati che le sono associate. QUando file esce dal suo scope, oltre a liberare la memoria a lei allocata, sono deallocati tutti i riferimenti connessi a quella variabile. -> modello RAI -> informazioni legate alla risorsa sono rilasciate quando la variabile esce dallo scope

- Rust offre una risposta funzionale al problema della modellazione degli errori basata sul tipo algebrico generico **Result<T,E>**
 - Modella l'unione di tutti i possibili risultati con successo **T** e tutti gli errori **E** che possono verificarsi nell'esecuzione di una funzione

```
enum Result<T, E> {  
    Ok(T),  
    Err(E),  
}
```

nel caso positivo, file contiene il puntatore al file aperto

why è di tipo io::Error ed è il dato che viene restituito da Err: specifica la tipologia di errore che restituito dall'errore

Ok contiene un dato che rappresento con la stringa s

```
fn read_file(name: &str)-> Result<String,io::Error> {  
    let r1 = File::open(name); // potrebbe non esistere  
    let mut file = match r1 {  
        Err(why) => return Err(why),  
        Ok(file) => file,  
    };  
    let mut s = String::new();  
    let r2 = file.read_to_string(&mut s);  
    match r2 {  
        Err(why) => Err(why),  
        Ok(_) => Ok(s),  
    }  
}
```



Non c'è la chiusura del file.
Perché?

Elaborare i risultati

- **Result<T,E>** mette a disposizione svariati metodi che consentono di accedere ai dati contenuti al suo interno
 - I metodi **is_ok(&self)->bool** e **is_err(&self)->bool** permettono, rispettivamente, di determinare se l'esito di un'operazione ha avuto successo o meno
 - I metodi **ok(self)-> Option<T>** e **err(self)-> Option<E>** consumano il risultato trasformandolo in un oggetto di tipo **Option<T>** piuttosto che **Option<E>**
 - **ok(self)** restituisce **Option<T>**, valorizzato con il dato in caso di successo (il dato risulta mosso da **Result**, che quindi diventa inaccessibile)
 - **err(self)** restituisce **Option<E>**, valorizzato con l'errore in caso di fallimento (anche in questo caso, con movimento)
 - Il metodo **map(self, op: F)->Result<U,E>** applica la funzione al valore contenuto nel risultato, se questo è ok, altrimenti lascia l'errore invariato
 - Il metodo **unwrap(self)->T** restituisce il valore contenuto, se è valido, ma invoca la macro **panic!(...)** se il risultato contiene un errore; in modo duale, il metodo **unwrap_err()->E** viene utilizzata per estrarre il valore di **errore** da un **Result** quando ci si aspetta che l'operazione sia fallita. Se l'operazione è invece riuscita, il programma andrà in *panic*.

```
fn divide(x: f64, y: f64) -> Result<f64, &'static str> {  
    if y == 0.0 {  
        Err("Impossibile dividere per zero")  
    } else {  
        Ok(x / y)  
    }  
}
```

Ignorare gli errori

- In alcune situazioni, può capitare che l'errore che potenzialmente viene restituito da una funzione non possa di fatto succedere, a conseguenza di quanto si è appena verificato nel corso dell'esecuzione
 - Oppure che il programmatore assuma che non sia necessario mettere in atto una strategia di contenimento e gestione dell'errore, lasciando semplicemente terminare il processo
- Il tipo **Result<T,E>** mette a disposizione i metodi **unwrap()** e **expect(...)** che restituiscono il valore di tipo **T**, se presente
 - Ed invocano la macro **panic!(...)** in caso di errore
 - **expect(...)** permette di indicare una stringa che sarà usata al posto del messaggio standard generato da **unwrap()**, nel caso si sia verificato un errore

```
fn divide(x: f64, y: f64) -> Result<f64, &'static str> {
```

```
    . . .
```

```
}
```

expect è equivalente all'unwrap ma passo un parametro e diventa il parametro del panic, quindi sto forzando qual è la stringa che viene stampata dal panic in caso di errore

```
fn main() {
```

```
    let dividend = 10.0;
```

```
    let divisor_success = 2.0;
```

```
    let divisor_error = 0.0;
```

unwrap spacchetta il risultato e lo restituisce dentro la variabile result_access

```
    let result_success = divide(dividend, divisor_success).unwrap();
```

```
    println!("{}", result_success);
```

```
    let result_error = divide(dividend, divisor_error).unwrap_err();
```

```
    println!("{}", result_error);
```

```
}
```

unwrap assume che il tipo Result sia sempre Ok. Se il risultato dell'unwrap non funziona e restituisce Err il programma va in panic



Per l'enum `Result<T, E>` il metodo `unwrap()` si comporta nel seguente modo:

- Se il `Result` è `Ok(T)`, estrae e restituisce il valore `T` contenuto all'interno della variante `Ok`.
- Se il `Result` è `Err(E)`, causa un panic (interruzione del programma) con un messaggio che indica che è stato chiamato `unwrap()` su un valore `Err`.

Il metodo `unwrap_err()` si comporta nel seguente modo:

- se il `Result` è `Err(e)`, il metodo restituisce il valore `e` contenuto all'interno della variante `Err`.
- se il `Result` è `Ok(t)` il metodo causa un panic (interruzione del programma) con un messaggio che indica che è stato chiamato `unwrap_err()` su un valore `Ok`.

```
fn divide(x: f64, y: f64) -> Result<f64, &'static str> {  
    . . .  
}  
  
fn main() {  
    let dividend = 10.0;  
    let divisor = 0.0;  
  
    let result = divide(dividend, divisor);  
  
    if result.is_ok() {  
        println!("Il risultato della divisione è: {}", result.unwrap() );  
    } else {  
        println!("Errore: {}", result.unwrap_err() );  
    }  
}
```

utilizza spesso questo metodo di `is_ok()`



Funzionamento di `ok()` e `err()`

Some porta dentro il dato
None indica che non c'è risultato

- `fn ok(self) -> Option<T>`
T è di tipo Some -> Some>T>
- Il metodo `ok()` converte un `Result<T, E>` in un `Option<T>`:
 - Se il `Result` è `Ok(val)`, il metodo `ok()` restituisce `Some(val)`. Il valore contenuto nella variante `Ok` viene "avvolto" in un `Some` nell'`Option` risultante
 - Se il `Result` è `Err(err)`, il metodo `ok()` restituisce `None`. Il valore di errore contenuto nella variante `Err` viene semplicemente scartato, e l'`Option` risultante indica l'assenza di un valore di successo.
- `fn err(self) -> Option<E>`
- Il metodo `err()` converte un `Result<T, E>` in un `Option<E>`:
 - Se il `Result` è `Err(err)`, il metodo `err()` restituisce `Some(err)`. Il valore contenuto nella variante `Err` viene "avvolto" in un `Some` nell'`Option` risultante
 - Se il `Result` è `Ok(val)`, il metodo `err()` restituisce `None`, scartando il valore di successo.

```
fn divide(x: f64, y: f64) -> Result<f64, &'static str> {
```

```
    . . .  
}
```

```
fn main() {
```

```
    let dividend = 10.0;
```

```
    let divisor_success = 2.0;
```

```
    let divisor_error = 0.0;
```

```
    let result_success = divide(dividend, divisor_success);
```

```
    let result_error = divide(dividend, divisor_error);
```

```
    println!("10 / 2");
```

```
    match result_success.ok() {
```

```
        Some(value) => println!("Il risultato della divisione è: {}", value),
```

```
        None => println!("Nessun risultato presente"),
```

```
    }
```

```
    match result_success.err() {
```

```
        Some(message) => println!("Errore: {}", message),
```

```
        None => println!("Nessun errore presente"), }
```

```
    println!("10 / 0");
```

```
    match result_error.ok() {
```

```
        Some(value) => println!("Il risultato della divisione è: {}", value),
```

```
        None => println!("Nessun risultato presente"), }
```

```
    match result_error.err() {
```

```
        Some(error_message) => println!("Errore: {}", error_message),
```

```
        None => println!("Nessun errore presente"), }
```

```
}
```



Funzionamento di `map()` e `map_err()`

la funzione lavora sul dato di successo di `Result` e quindi riesce a fornire un risultato.

`map` permette quindi di fare un'operazione sul dato qualora esso esista. Il dato su cui lavora `map` è stato consumato, per cui non posso chiamare due volte `F`

```
pub fn map<U, F>(self, op: F) -> Result<U, E>  
where F: FnOnce(T) -> U
```

```
pub fn map_err<F, O>(self, op: O) -> Result<T, F>  
where O: FnOnce(E) -> F
```

`map_err` invece lavora solo sull'errore

- Il metodo `map()` si comporta nel modo seguente:
 - se il `Result` è `Ok`, prende una closure che viene chiamata con il valore contenuto in `Ok`. Il risultato della closure viene avvolto in un nuovo `Ok`
 - Se il `Result` è `Err(e)`, la closure non viene chiamata e `map()` restituisce lo stesso `Err(e)`
- Il metodo `map_err()` si comporta nel modo seguente:
 - se il `Result` è `Ok`, ~~la closure non viene chiamata e `map_err()`~~ restituisce lo stesso `Ok`
 - se il `Result` è `Err(e)`, la closure viene chiamata con il valore contenuto in `Err(e)`. Il risultato della closure viene avvolto in un nuovo `Err` che contiene un nuovo valore di errore: `Err(op(e))`.
 - Si noti che:
 - `O` è il tipo della closure
 - `F` è il tipo del nuovo valore di errore che la closure restituirà

```
fn divide(x: f64, y: f64) -> Result<f64, &'static str> {  
    . . .  
}
```

```
fn to_percentage(value: f64) -> f64 {  
    value * 100.0  
}
```

```
fn main() {  
    let dividend = 12.0;  
    let divisor = 55.0;  
  
    // Utilizzando map() per convertire il risultato della divisione in percentuale  
    let result = divide(dividend, divisor).map(to_percentage);  
  
    match result {  
        Ok(value) => println!("Risultato della divisione in percentuale è:{:.1}%", value),  
        Err(error_message) => println!("Errore: {}", error_message),  
    }  
}
```



```
fn main() {  
    fn parse_and_add_one(s: &str) -> Result<i32, String> {  
        s.parse::<i32>()  
            .map(|n| n + 1) // map() viene chiamato solo se parse() restituisce Ok  
            .map_err(|_| format!("Impossibile convertire '{}'' in un intero", s))  
    }  
  
    let success = parse_and_add_one("10");  
    println!("Successo: {:?}", success);  
    // Output: Successo: Ok(11)  
  
    let failure = parse_and_add_one("abc");  
    println!("Fallimento: {:?}", failure);  
    // Output: Fallimento: Err("Impossibile convertire 'abc' in un intero")  
}
```

12:36 21-04-2026

Text

Text

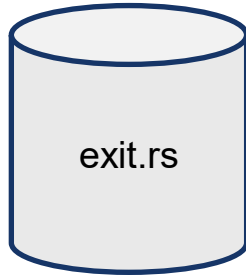


Gestire gli errori

- Se l'invocazione di una funzione restituisce un valore di tipo `Result` contenente un errore, occorre mettere in atto una strategia di gestione
 - Terminare, in modo ordinato, il programma (rilasciando ciò che può essere rilasciato e marcando come invalidi risultati parziali che potrebbero essere stati costruiti)
 - Ritentare l'esecuzione, evitando di entrare in un loop infinito (tipico dell'applicazioni distribuite, il tempo di attesa è incrementale, *exponential backoff*, la prima volta riprovo dopo 1 unità di tempo, poi dopo 2, poi dopo 4, poi dopo 8 e così' via, fino ad un massimo *time-out* dopo cui viene abortito)
 - Registrare un messaggio nel log e propagare l'errore al chiamante (si lascia traccia dell'errore, specificando la severità dell'errore che si è verificato e informando su *Who* (chi stava eseguendo), *What* (che cosa si è rotto), *Why* (perché si è rotto), *When* (quando si è rotto), *Where* (dove si è rotto))
 - Propagare *tout court* l'errore al chiamante, delegando ad esso la corrispondente responsabilità
 - Altro...
- Sebbene terminare il programma possa apparire facile (è sufficiente invocare la funzione `std::process::exit(code: i32) -> !`), farlo in modo pulito può essere complesso
 - Occorre infatti garantire che non vengano lasciati oggetti persistenti (come file o altre risorse di sistema) in stati non coerenti: sconsigliatissimo!
 - Per questo motivo, Rust offre la macro `panic!(...)`

```
use std::process;

fn main() {
    println!("Il programma sta per terminare con successo.");
    process::exit(100);
    // Qualsiasi codice dopo exit non verrà eseguito.
    println!("Questo non verrà stampato.");
}
```



vengono chiusi tutti i processi thread oltre al thread principale

Il programma sta per terminare con successo.

error: process didn't exit successfully: `target\debug\untitled38.exe` (exit code: 100)

Process finished with exit code 100

Panic

- Serve per manifestare il fatto che il programma ha raggiunto un punto di incoerenza oltre il quale non occorre proseguire per non fare danni
- In alcune situazioni, lo stato del programma risulta così compromesso che non è pensabile eseguire azioni di ripristino: questo spesso è dovuto alla presenza di errori logici nel programma stesso
 - Accesso ad array al di fuori dei relativi limiti
 - Divisioni di interi per zero
 - Fallimenti di asserzioni
- In queste situazioni, Rust offre la macro **panic!(...)** che accetta argomenti simili a quelli offerti dalla macro **println!(...)** per formulare un messaggio di errore
 - L'effetto dell'invocazione di questa macro è la contrazione dello stack (come nel caso delle eccezioni C++), la distruzione (via **.drop()**) delle variabili che implementano il tratto **Drop** fino alla terminazione del thread corrente
 - Se il thread in cui è stato invocato **panic!(...)** è il thread principale dell'applicazione, il processo termina con un codice di errore non nullo, altrimenti **continua**

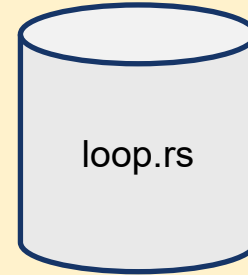
```
fn main() {  
    let numbers = vec![1, 2, 3, 4, 5];  
    let index = 5;  
  
    if index < numbers.len() {  
        let value = numbers[index];  
        println!("Il valore all'indice {} è: {}", index, value);  
    } else {  
        panic!("Tentativo di accedere a un indice non valido.");  
    }  
}
```



Ripetere l'esecuzione

12:45 21-04-2026

```
fn main() {  
    loop {  
        println!("Enter the name of the file to open:");  
  
        let mut filename = String::new();  
        io::stdin().read_line(&mut filename);  
  
        let filename = filename.trim();  
  
        match File::open(filename) {  
            Ok(file) => {  
                println!("Successfully opened {}", filename);  
                // Esegui altre operazioni sul file aperto qui se necessario  
                break; // Esci dal ciclo se l'apertura ha avuto successo  
            },  
            Err(err) => {  
                println!("Error opening {}: {}", filename, err);  
                println!("Please try again.");  
            }  
        }  
    }  
}
```



Propagare gli errori: operatore **try**

il ? viene messo a valle di una funzione per la gestione del Result. Se esso è Ok il ? estrae il contenuto dell'Ok

Se il result è un Err, interrompe l'esecuzione e restituisce al chiamante Err.

- Nella maggior parte delle funzioni in cui si verifica un errore, non si sa quale strategia mettere in atto per ripristinare lo stato del programma
 - Si può facilmente ovviare a questo problema restituendo, a propria volta, un oggetto di tipo **Result<T,E>**
 - Questo porta, tuttavia, a costrutti alquanto complessi, con espressioni di controllo difficili da decodificare
- Per semplificare la sintassi, Rust offre l'operatore **?** che può essere applicato a qualunque espressione che produce un valore di tipo **Result<T,E>**
 - Se il valore risulta essere **Ok(v)**, l'operatore restituisce il valore **v** racchiuso nell'enum
 - Se, invece, risulta essere **Err(e)**, la funzione corrente **termina immediatamente**, ritornando il valore che incapsula l'errore
- Perché questo comportamento possa funzionare, occorre che la funzione in cui è usato l'operatore **?** ritorni un oggetto di tipo **Result<U,E>**
 - E che il tipo dell'errore ritornato sia compatibile con l'errore **E** che proviene dalla funzione chiamata
 - Se ci sono più ? i tipi degli errori devono essere congruenti.

Propagare gli errori



- L'utilizzo dell'operatore `?` permette di ottenere una sintassi molto più compatta (**syntax sugar**), evidenziando il comportamento della funzione lungo il cammino principale
 - E demandando al compilatore la scrittura delle clausole `if / match` necessarie a valutare il comportamento da adottare

```
fn parse_integer(text: &str) -> Result<i32, std::num::ParseIntError> {  
    // L'operatore ? gestisce la possibile generazione di un errore  
    // durante il parsing della stringa
```

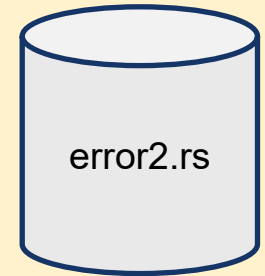
```
    let number: i32 = text.parse()?;
```

```
    Ok(number)
```

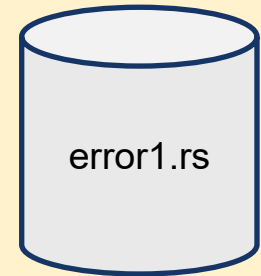
```
}
```

il ? ha il ruolo di fare il match sull'Ok in caso di Ok ed estrae il dato. Quindi in caso positivo restituisce Ok, non i32
Se al contrario restituiva un Errore il punto interrogativo ritorna l'errore interrompendo l'esecuzione.
devo quindi creare una compatibilità del valore di ritorno restituito dalla funzione che usa il punto interrogativo

```
fn main() {  
    match parse_integer("42") {  
        Ok(number) => println!("Parsed integer: {}", number),  
        Err(err) => eprintln!("Error: {}", err),  
    }  
}
```



```
fn read_file_contents(filename: &str) -> Result<String, io::Error> {  
    let mut file = File::open(filename)? ;  
    // Apre il file e gestisce gli errori con l'operatore ?  
    let mut contents = String::new();  
    file.read_to_string(&mut contents)? ;  
    // Legge il contenuto del file e gestisce gli errori con l'operatore ?  
    Ok(contents) // Restituisce il contenuto del file se tutto è andato bene  
                // di tipo Result  
}  
  
fn main() {  
    match read_file_contents("prova.txt") {  
        Ok(contents) => println!("Contenuto del file: {}", contents),  
        Err(err) => println!("Errore: {}", err),  
    }  
}
```



Altri modi di esprimere il fallimento

- In alcune situazioni, può essere sufficiente distinguere se la computazione ha prodotto il proprio risultato, oppure indicare che non è stato possibile completarla
 - Per questi casi esiste il tipo **Option<T>** che racchiude due alternative:
 - **Some<T>**, usata per descrivere il risultato atteso
 - **None**, che indica l'assenza di risultato, senza descriverne le ragioni
- L'operatore **?** può essere applicato anche al tipo **Option<T>**
- La funzione in cui viene adottato deve avere tipo di ritorno **Option<U>**, con **U** qualsiasi.

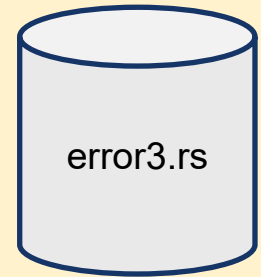
```
fn get_first_element(numbers: Vec<i32>) -> Option<i32> {
    let first = numbers.first()? ;
    Some(*first)
}

fn main() {
    let numbers = vec![1, 2, 3, 4, 5];
    let first_element = get_first_element(numbers);

    match first_element {
        Some(n) => println!("Il primo elemento è: {}", n),
        None => println!("La lista è vuota!"),
    }

    let empty_numbers: Vec<i32> = vec![];
    let first_element_empty = get_first_element(empty_numbers);

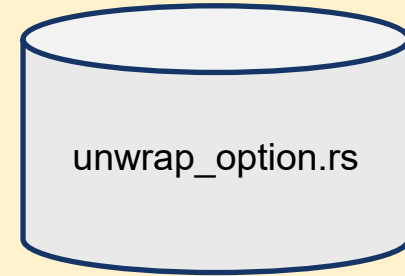
    match first_element_empty {
        Some(n) => println!("Il primo elemento è: {}", n),
        None => println!("La lista è vuota!"),
    }
}
```



```
fn get_first_word(sentence: &str) -> Option<&str> {  
    let first_space = sentence.find(' ')? ;  
    Some(&sentence[..first_space])  
}  
  
fn main() {  
    let sentence = "Hello,world!";  
  
    match get_first_word(sentence) {  
        Some(word) => println!("Il primo parola è: {}", word),  
        None => println!("La stringa è vuota o non contiene spazi!"),  
    }  
    match get_first_word("ciao mamma") {  
        Some(word) => println!("Il primo parola è: {}", word),  
        None => println!("La stringa è vuota o non contiene spazi!"),  
    }  
}
```



```
fn main() {  
    // Un Option che contiene un valore (Some)  
    let valore_presente: Option<i32> = Some(10);  
  
    // Usiamo unwrap() per ottenere il valore.  
    let numero = valore_presente.unwrap();  
    println!("Il numero è: {}", numero); // Output: Il numero è: 10  
  
    // Un Option che non contiene un valore (None)  
    let valore_assente: Option<String> = None;  
  
    // Se chiamiamo unwrap() su un Option che è None, il programma andrà in panic.  
    // Il seguente codice causerà un crash:  
    // let testo = valore_assente.unwrap();  
    // println!("Il testo è: {}", testo);  
  
    // Per evitare il panic, dovremmo sempre verificare se l'Option è Some prima di chiamare unwrap()  
    if valore_assente.is_some() {  
        let testo = valore_assente.unwrap();  
        println!("Il testo è: {}", testo);  
    } else {  
        println!("Il valore di testo è assente."); // Output: Il valore di testo è assente.  
    }  
}
```



Per l'enum `Option<T>` il metodo `unwrap()` si comporta nel seguente modo:

- se l'`Option` è `Some(T)`, il metodo restituisce il valore `T` contenuto all'interno della variante `Some`
- se l'`Option` è `None` il metodo causa un panic (interruzione del programma) con un messaggio che indica che è stato chiamato `unwrap_err()` su un valore `None`.

Il metodo `unwrap_err()` non può essere applicato ad un `Option<T>`.

Operatore ? nel main

- L'operatore ? può essere usato solo in funzioni che ritornano un tipo di `Result<T,E>` oppure `Option<T>`
- Può essere usato nel main solo se dichiariamo che restituisce un valore di ritorno di tipo `Result<T,E>`.

si applica ad un metodo che restituisce un `Result`. Se il valore di ritorno è un `Err`, l'esecuzione del metodo viene interrotta e si ritorna un valore di ritorno al chiamante.

ci deve essere un legame tra gli `Err`

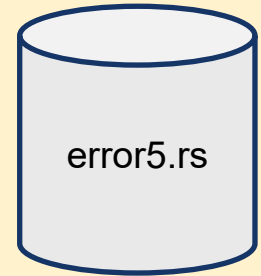
se introduco il ? in un meodo nel main si può fare ma devo cambiare la signature del main, che deve contenere o un `Result` o un `Option`

```
fn divide(x: f64, y: f64) -> Result<f64, String> {  
    if y == 0.0 {  
        return Err(String::from("Divisione per zero"));  
    }  
    Ok(x / y)  
}
```

modifica del main

```
fn main() -> Result<(), String> {  
    let result = divide(10.0, 5.0)? ;  
    println!("Il risultato della divisione è: {}", result);  
  
    let result2 = divide(10.0, 0.0)? ;  
    println!("Il risultato della divisione è: {}", result2);  
  
    Ok(())  
}
```

poichè la funzione in se non ritorna niente, devo usare questa notazione



eterogenei = non dello stesso tipo



posso gestire questo tramite gli oggetti tratto

Propagare errori eterogenei

- Quando una funzione produce diverse tipologie di errore, è necessario propagare i diversi errori così da poter specializzare le contromisure
 - Rust offre diversi modi per propagare errori eterogenei, la scelta spetta al programmatore sulla base delle sue esigenze
- La libreria standard mette a disposizione l'implementazione generica del tratto **From**, che converte **qualsiasi valore che implementi il tratto Error** in **Box<dyn error::Error>**
 - Durante la conversione vengono perse le informazioni sul tipo dell'errore
 - Si può risalire allo specifico errore tramite l'utilizzo del **downcast_ref()**
 - La conversione può avvenire in maniera implicita attraverso l'utilizzo dell'operatore **?**

```

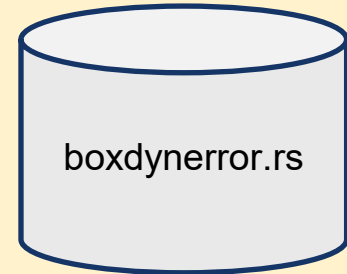
fn sum_file(path: &Path) -> Result<i32, Box<dyn error::Error> > {
    let mut file = File::open(path) ? ; // io::Error -> Box<dyn error::Error>
    let mut contents = String::new();
    file.read_to_string(&mut contents) ? ; // io::Error -> Box<dyn error::Error>
    let mut sum = 0;
    for line in contents.lines() {
        sum += line.parse::<i32>() ? ; // ParseIntError -> Box<dyn error::Error>
    }
    Ok(sum)
}

fn handle_sum_file_errors(path: &Path) {
    match sum_file(path) {
        Ok(sum) => println!("sum is {}", sum),
        Err(err) => {
            if let Some(e) = err.downcast_ref::<io::Error>()
                {
                    //tratto io::Error
                    println!("Errore di I/O nella gestione del file: {}", e);
                }
            else if let Some(e) = err.downcast_ref::<ParseIntError>()
                {
                    //tratto ParseIntError
                    println!("Errore nell'elaborazione del file: {}", e);
                }
            else { unreachable!(); } //non può capitare
        }
    }
}

```

gode del tratto Error

serve per ottenere la tipologia di errore



Macro che se viene invocata
scatena un panic

Operatore ? nel main con errori eterogenei

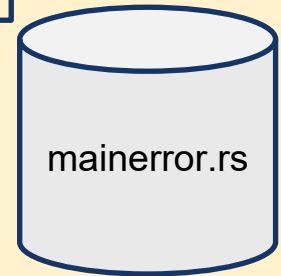
```
use std::error::Error;
```

```
fn divide(x: f64, y: f64) -> Result<f64, String> {  
    if y == 0.0 {  
        return Err(String::from("Divisione per zero"));  
    }  
    Ok(x / y)  
}
```

Lo si può leggere come:
"ogni tipo di possibile errore"

```
fn main() -> Result<(), Box<dyn Error> > {  
    let result = divide(10.0, 5.0)? ;  
    println!("Il risultato della divisione è: {}", result);  
  
    let text = "42";  
    let number: i32 = text.parse()?;  
    println!("Il numero è: {}", number);  
  
    Ok(())  
}
```

il tratto error deve essere implementato dalle istruzioni possono generare dei tipi di errore
tutte le funzioni standard implementano il tratto Error



Propagare errori eterogenei

definisce un'interfaccia comune che mostra i vari tipi di errori

se prima abbiamo usato gli oggetti-tratto, ora stiamo usando un tipo ad hoc, ovvero un'enumerazione che prevede tra le varianti tutti i possibili tipi di errori

- Per propagare errori eterogenei senza forzare il sistema dei tipi è possibile implementare un tipo di errore custom
 - Tutti gli errori custom devono implementare il tratto **Error** e conseguentemente anche i tratti **Debug** e **Display**
 - L'utilizzo di un **enum** permette di racchiudere i diversi tipi di errore da gestire successivamente tramite l'utilizzo del costrutto **match**
 - E' necessario implementare il tratto **From** per convertire i diversi errori nel tipo custom da propagare

per implementare il ? bisogna fare in modo che il tipo di personalizzato implementi certi tratti, come **Debug** e **Display**

il tratto **Display** richiede che abbia un'implementazione specializzata per l'enum e bisogna definire il tratto **From** per permettere la conversione tra tutte le varianti di errore verso il tipo **Error**

```
#[derive(Debug)]
enum SumFileError {
    Io(io::Error),
    Parse(ParseIntError),
}
```

importante per fare il progetto
serve per identificare tutti i possibili errori,
anche errori di gestione di progetto. Ho un tipo
di errore unico in cui convergono tutti i possibili tipi di errori

Propagare errori eterogenei

Da ripetersi per tutte
le possibili varianti:
boilerplate code

```
impl From<io::Error> for SumFileError {  
    fn from(err: io::Error) -> Self { SumFileError::Io(err) }  
}  
impl From<ParseIntError> for SumFileError {  
    fn from(err: ParseIntError) -> Self { SumFileError::Parse(err) }  
}  
impl fmt::Display for SumFileError {  
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {  
        match self {  
            SumFileError::Io(err) => write!(f, "IO error: {}", err),  
            SumFileError::Parse(err) => write!(f, "Parse error: {}", err),  
        }  
    }  
}  
impl error::Error for SumFileError {  
    fn source(&self) -> Option<&(dyn error::Error + 'static)> {  
        Some(match self {  
            SumFileError::Io(err) => err,  
            SumFileError::Parse(err) => err,  
        })  
    }  
}
```



Propagare errori eterogenei

```
fn sum_file(path: &Path) -> Result<i32, SumFileError> {
    let mut file = File::open(path)?;
    let mut contents = String::new();
    file.read_to_string(&mut contents)?;
    let mut sum = 0;
    for line in contents.lines() {
        sum += line.parse::<i32>()?;
    }
    Ok(sum)
}

fn handle_sum_file_errors(path: &Path) {
    match sum_file(path) {
        Ok(sum) => println!("the sum is {}", sum),
        Err(SumFileError::Io(err)) => { println!("{}", err)},
        Err(SumFileError::Parse(err)) => { println!("{}", err)},
    }
}

fn main() {
    let path = Path::new("file.txt");
    handle_sum_file_errors(&path);
}
```

Propagare errori eterogenei

- Un grosso aiuto all'implementazione di tipi che implementano il tratto `Error` viene dal crate **thiserror**
 - Esso offre una implementazione della macro **derive** specializzata per il tratto `Error`
- Definendo un tipo (enum, struct) preceduto dall'attributo **#[derive(Error, Debug)]** si ottiene l'implementazione automatica di questi due tratti
 - L'attributo **#[error("Messaggio con formato")]** posta di fronte alle singole voci enumerative o all'intera struct genera l'implementazione del tratto **Display**
 - L'attributo **#[from]** posto di fronte ad un campo il cui tipo implementa il tratto `Error` genera l'implementazione del tratto **From** a partire dal tipo del campo
- Questo approccio è particolarmente adatto nella creazione di librerie, dove si vogliono rendere i possibili errori parte della definizione del contratto (API)

Propagare errori eterogenei

```
[dependencies]  
thiserror = "2.0"
```

```
#[derive(Error, Debug)]  
enum SumFileError {  
  
    #[error("IO error {0}")]  
    Io(#[from] io::Error),  
  
    #[error("Parse error {0}")]  
    Parse(#[from] ParseIntError),  
}
```

comandi formattati con il simbolo #

importante per progetto

```
#[derive(Error, Debug)]
pub enum SumFileError {
    #[error("IO Error {0}")]
    Io(#[from] io::Error),
    #[error("Parse Error {0}")]
    Parse(#[from] ParseIntError),
}

fn sum_file(path: &Path) -> Result<i32, SumFileError> {
    let mut file = File::open(path)?;
    let mut contents = String::new();
    file.read_to_string(&mut contents)?;
    let mut sum = 0;
    for line in contents.lines() { sum += line.parse::<i32>()?; }
    Ok(sum)
}

fn handle_sum_file_errors(path: &Path) {
    match sum_file(path) {
        Ok(sum) => println!("the sum is {}", sum),
        Err(SumFileError::Io(err)) => { println!("{}", err)},
        Err(SumFileError::Parse(err)) => { println!("{}", err)},
    }
}

fn main() {
    let path = Path::new("file.txt");
    handle_sum_file_errors(&path);
}
```



Propagare errori eterogenei

- Il crate **anyhow** definisce l'oggetto-tratto **anyhow::Error** che semplifica la gestione idiomatica degli errori
 - Si può usare il tipo **anyhow::Result<T>** per incapsulare il valore di ritorno di una funzione che può fallire (alias di **Result<T, anyhow::Error>**).
- Questo tipo offre un'implementazione automatica del tratto **From<T: Error>**, il che permette di utilizzare la notazione basata sull'operatore **?** per propagare l'errore ritornato
 - Quando viene generato un errore è possibile aggiungere una descrizione che contestualizza ciò che è successo tramite i metodi **context(...)** e **with_context(...)**
 - Il messaggio di errore associato verrà sostituito dalla stringa indicata seguita dalla causa originale
- Questo crate interopera correttamente con **thiserror** e risulta adatto nella scrittura di codice applicativo, piuttosto che di librerie
 - In questo caso, la semplicità di scrittura del codice domina rispetto al controllo dell'informazione contenuta nell'errore generato, tenuto conto che sarà interpretato da persone

Propagare errori eterogenei

```
fn sum_file(path: &Path) -> anyhow::Result<i32> {
    let mut file = File::open(path).with_context(|| format!("Missing path {}", path)) ? ;

    let mut contents = String::new();
    file.read_to_string(&mut contents).context("File read error") ? ;
    let mut sum = 0;
    for line in contents.lines() {
        sum += line.parse::<i32>().with_context(|| format!("Not a number: {}", line)) ? ;
    }
    Ok(sum)
}

fn handle_sum_file_errors(path: &Path) {
    match sum_file(path) {
        Ok(sum) => println!("sum is {}", sum),
        Err(err) => {
            if let Some(e) = err.downcast_ref::<io::Error>() {...} //tratto io::Error
            else if let Some(e) = err.downcast_ref::<ParseIntError>() {...} //tratto ParseIntError
            else { unreachable!(); } //non può capitare
        }
    }
}
```



Glossario

- Operatore try ? : Operatore suffisso che si applica ai valori di tipo `Result<T,E>` e a `Option<T>` per propagare gli errori in modo semplificato e conciso
- Syntax sugar: modifica della sintassi che rende le cose più facili da leggere o scrivere senza però aggiungere alcuna funzionalità al linguaggio (ossia un modo più comodo o carino di scrivere codice che potrebbe essere espresso in modo più verboso o meno intuitivo). Il compilatore si occuperà poi di togliere lo zucchero e tradurre il codice nella forma sottostante più complessa
- Boilerplate code: sezioni di codice ripetitive che devono essere inserite con poca o nessuna modifica